

BÁO CÁO BTVN 2: TÍCH PHÂN NHIỀU CHIỀU VÀ PHƯƠNG PHÁP MONTE CARLO

Môn: Vật lý tính toán

Computational Physics

1 Nội dung bài tập

Bài tập gồm ba phần chính:

1. **Thực hành 1a:** viết thuật toán Monte Carlo để tính tích phân nhiều chiều

$$I_D = \int_0^1 dx_1 \int_0^1 dx_2 \cdots \int_0^1 dx_D (x_1^2 + x_2^2 + \cdots + x_D^2),$$

khảo sát cho các trường hợp nhiều chiều và ước lượng số điểm ngẫu nhiên cần thiết để đạt độ chính xác cỡ 10^{-4} .

2. **Thực hành 1b:** tính gần đúng tích phân

$$I_{10} = \int_0^1 dx_1 \int_0^1 dx_2 \cdots \int_0^1 dx_{10} (x_1 + x_2 + \cdots + x_{10})^2,$$

rồi so sánh phương pháp tổng Riemann thông thường với phương pháp Monte Carlo. Nghiệm giải tích đã cho là $155/6$.

3. **Thực hành MC 2:**

- a) tính khối lượng của khối khí trong lập phương đơn vị với mật độ

$$\rho(x, y, z) = x^2 + y^2 + z^2;$$

- b) tính thể tích phần giao giữa hình cầu bán kính $R = 1$ tâm tại gốc tọa độ và hình trụ bán kính $r = 0.5$ dọc theo trục z .

2 Cơ sở phương pháp

2.1 Công thức Monte Carlo cho tích phân nhiều chiều

Xét tích phân trên miền $\Omega \subset \mathbb{R}^D$

$$I = \int_{\Omega} f(\mathbf{x}) d\mathbf{x}.$$

Nếu lấy ngẫu nhiên N điểm $\mathbf{x}_k$ phân bố đều trên miền Ω , tích phân được xấp xỉ bởi

$$I \approx V_{\Omega} \frac{1}{N} \sum_{k=1}^N f(\mathbf{x}_k),$$

trong đó V_{Ω} là thể tích của miền lấy mẫu.

Ưu điểm của Monte Carlo là phương pháp này phù hợp cho tích phân nhiều chiều do chạy nhanh hơn phương pháp tổng Riemann.

2.2 Ước lượng sai số Monte Carlo

Gọi giá trị trung bình mẫu là

$$\bar{f} = \frac{1}{N} \sum_{k=1}^N f(\mathbf{x}_k).$$

Khi đó sai số chuẩn của tích phân có thể ước lượng từ phương sai mẫu:

$$\sigma_I \approx V_\Omega \sqrt{\frac{1}{N(N-1)} \sum_{k=1}^N (f(\mathbf{x}_k) - \bar{f})^2}.$$

Trong code, biểu thức này được tính lại dưới dạng tương đương bằng cách lưu đồng thời $\sum f$ và $\sum f^2$.

3 Phân tích chương trình

3.1 Khối 1: Hàm Monte Carlo tổng quát cho tích phân nhiều chiều

```
1 def tich_phan_monte_carlo_nhieu_chieu(a, b, N, N_D, hamso):  
2     S = 0  
3     for i in range(N):  
4         x = []  
5         for j in range(N_D):  
6             x.append(np.random.uniform(a[j], b[j]))  
7         S = S + hamso(x)  
8  
9     tich = 1  
10    for j in range(N_D):  
11        tich = tich * (b[j] - a[j])  
12  
13    ketqua = tich * S/N  
14    return ketqua
```

Listing 1: Hàm Monte Carlo tổng quát dùng cho tích phân nhiều chiều

Giải thích:

Hàm trên thực hiện đúng công thức Monte Carlo cơ bản:

- sinh N điểm ngẫu nhiên trong miền hộp chữ nhật $[a_1, b_1] \times \cdots \times [a_D, b_D]$;
- tính giá trị hàm dưới dấu tích phân tại mỗi điểm;
- lấy trung bình cộng các giá trị đó;
- nhân với thể tích miền tích phân để thu được giá trị gần đúng của tích phân.

3.2 Khối 2: Hàm ước lượng sai số Monte Carlo

```
1 def MC_nhieu_chieu_tinh_saiso(a, b, N, N_D, hamso):  
2     tong_f = 0.0  
3     tong_f2 = 0.0  
4  
5     for i in range(N):  
6         x = []  
7         for j in range(N_D):  
8             x.append(np.random.uniform(a[j], b[j]))  
9  
10        fx = hamso(x)  
11        tong_f += fx  
12        tong_f2 += fx**2  
13
```

```
14 V = 1.0
15 for j in range(N_D):
16     V *= (b[j] - a[j])
17
18 f_tb = tong_f / N
19 ketqua = V * f_tb
20
21 if N > 1:
22     phuong_sai_mau = (tong_f2 - N * f_tb**2) / (N - 1)
23     sai_so_uoc_luong = V * np.sqrt(phuong_sai_mau / N)
24 else:
25     sai_so_uoc_luong = 0.0
26
27 return ketqua, sai_so_uoc_luong
```

Listing 2: Hàm Monte Carlo có kèm ước lượng sai số

Giải thích:

Hàm này ngoài việc trả về giá trị tích phân còn trả về sai số chuẩn ước lượng. Đây là phần quan trọng đối với bài 1a vì đề bài yêu cầu xác định số mẫu N cần thiết để đạt độ chính xác cỡ 10^{-4} .

3.3 Khối 3: Tổng Riemann nhiều chiều bằng đệ quy

```
1 def tinh_tong_Riemann(a, b, N, N_D, hamso):
2     delta_h = []
3     for j in range(len(a)):
4         delta_h.append((b[j] - a[j]) / N)
5     dV = 1
6     for j in range(len(a)):
7         dV = dV * delta_h[j]
8
9     S = 0.0
10    x = [0.0] * N_D
11
12    def de_quy(chieu):
13        nonlocal S
14
15        if chieu == N_D:
16            S += hamso(x)
17            return
18
19        for i in range(N):
20            x[chieu] = a[chieu] + i * delta_h[chieu]
21            de_quy(chieu + 1)
22
23    de_quy(0)
24
25    return S * dV
```

Listing 3: Hàm tổng Riemann nhiều chiều

Giải thích:

Phương pháp tổng Riemann trong nhiều chiều cần duyệt toàn bộ các nút lưới. Nếu mỗi chiều chia thành N đoạn thì tổng số điểm lưới là N^D . Với $D = 10$, chi phí tính toán tăng rất nhanh. Trong bài này, khi chọn $N = 6$ cho mỗi chiều thì số điểm lưới đã là

$$6^{10} = 60,466,176.$$

Đây là nguyên nhân chính làm thời gian chạy của phương pháp này lớn hơn rõ rệt so với Monte Carlo.

3.4 Khối 4: Hàm chỉ thị dùng để tính thể tích giao nhau

```
1 def hamso_Nchieu_MC2_A(x):  
2     if x[0]**2 + x[1]**2 + x[2]**2 <= 1 and x[0]**2 + x[1]**2 <= 0.25:  
3         return 1  
4     else:  
5         return 0
```

Listing 4: Hàm chỉ thị cho bài toán thể tích phần giao

Giải thích:

Trong bài toán hình học, ta đưa việc tính thể tích về tích phân của hàm chỉ thị. Nếu điểm (x, y, z) đồng thời nằm trong hình cầu và trong hình trụ thì hàm chỉ thị nhận giá trị 1, ngược lại nhận giá trị 0. Trung bình Monte Carlo của hàm chỉ thị chính là tỉ phần thể tích của miền cần tìm trong hộp lấy mẫu.

4 Kết quả và thảo luận

4.1 Thực hành 1a: Tích phân của tổng bình phương trên siêu lập phương đơn vị

Hàm dưới dấu tích phân là

$$f(\mathbf{x}) = x_1^2 + x_2^2 + \cdots + x_D^2.$$

Do miền tích phân là siêu lập phương đơn vị, nghiệm giải tích có thể tính ngay:

$$I_D = \int_{[0,1]^D} \left(\sum_{i=1}^D x_i^2 \right) d\mathbf{x} = \sum_{i=1}^D \int_0^1 x_i^2 dx_i = \frac{D}{3}.$$

Vì vậy:

$$I_3 = 1, \quad I_4 = \frac{4}{3}, \quad I_{10} = \frac{10}{3}.$$

4.1.1 Kết quả chạy Monte Carlo

Từ file output đã chạy, kết quả thu được như Bảng 1.

Bảng 1: Kết quả bài 1a với Monte Carlo cho các trường hợp nhiều chiều

D	N	Kết quả MC	Nghiệm giải tích	Sai số tuyệt đối	Thời gian chạy (s)
3	10^6	0.999019692487	1.000000000000	0.000980307513	15.367670874
4	10^6	1.332944497497	1.333333333333	0.000388835836	19.368983514
10	10^6	3.333908441127	3.333333333333	0.000575107794	46.963117374

Kết quả cho thấy Monte Carlo cho giá trị gần đúng tốt ngay cả trong trường hợp số chiều tăng lên. Thời gian chạy tăng theo số chiều do mỗi lần đánh giá hàm cần cộng nhiều biến hơn, nhưng mức tăng vẫn chậm hơn rất nhiều so với phương pháp lưới tensor-product.

4.1.2 Ước lượng số mẫu cần thiết để đạt độ chính xác cỡ 10 mũ trừ 4

Từ file khảo sát sai số đã chạy, trong khoảng giá trị N được thử nghiệm, sai số ước lượng vẫn chưa giảm xuống dưới 10^{-4} cho các trường hợp đã khảo sát. Điều này phù hợp với quy luật hội tụ của Monte Carlo:

$$\sigma_I \propto \frac{1}{\sqrt{N}}.$$

Đối với bài này, có thể ước lượng trực tiếp bằng phương sai lý thuyết. Với biến ngẫu nhiên $X \sim U(0, 1)$:

$$\mathbb{E}[X^2] = \frac{1}{3}, \quad \mathbb{E}[X^4] = \frac{1}{5},$$

$$\text{Var}(X^2) = \frac{1}{5} - \frac{1}{9} = \frac{4}{45}.$$

Vì các biến độc lập,

$$\text{Var}\left(\sum_{i=1}^D X_i^2\right) = D \cdot \frac{4}{45}.$$

Do miền tích phân có thể tích bằng 1, sai số chuẩn Monte Carlo được xấp xỉ bởi

$$\sigma_I \approx \sqrt{\frac{4D}{45N}}.$$

Đặt $\sigma_I \leq 10^{-4}$, ta thu được

$$N \gtrsim \frac{4D}{45 \times 10^{-8}}.$$

Bảng 2: Ước lượng lý thuyết số mẫu để đạt sai số cỡ 10^{-4} trong bài 1a

D	Công thức ước lượng	N xấp xỉ
3	$\frac{12}{45 \times 10^{-8}}$	2.67×10^7
4	$\frac{16}{45 \times 10^{-8}}$	3.56×10^7
10	$\frac{40}{45 \times 10^{-8}}$	8.89×10^7

Như vậy, muốn đạt độ chính xác cỡ 10^{-4} thì số mẫu cần thiết là hàng chục triệu điểm. Đây cũng là lý do vì sao trong file khảo sát sai số, với vài triệu điểm ngẫu nhiên vẫn chưa đạt được ngưỡng yêu cầu.

4.2 Thực hành 1b: So sánh tổng Riemann và Monte Carlo cho tích phân 10 chiều

Xét tích phân

$$I_{10} = \int_{[0,1]^{10}} (x_1 + x_2 + \dots + x_{10})^2 d\mathbf{x}.$$

Đặt $S = x_1 + x_2 + \dots + x_{10}$, khi đó

$$S^2 = \sum_{i=1}^{10} x_i^2 + 2 \sum_{i < j} x_i x_j.$$

Lấy tích phân từng hạng tử:

$$\int_{[0,1]^{10}} x_i^2 d\mathbf{x} = \frac{1}{3}, \quad \int_{[0,1]^{10}} x_i x_j d\mathbf{x} = \frac{1}{4}.$$

Vì có 10 hạng tử kiểu x_i^2 và $\binom{10}{2} = 45$ hạng tử kiểu $x_i x_j$, nên

$$I_{10} = 10 \cdot \frac{1}{3} + 2 \cdot 45 \cdot \frac{1}{4} = \frac{10}{3} + \frac{45}{2} = \frac{155}{6} \approx 25.833333333333.$$

4.2.1 Kết quả so sánh

Trong chương trình đã chạy:

- tổng Riemann dùng $N_{\text{RM}} = 6$ cho mỗi chiều;

- Monte Carlo dùng $N_{MC} = 10^4$ điểm ngẫu nhiên.

Kết quả thu được như Bảng 3.

Bảng 3: So sánh tổng Riemann và Monte Carlo cho bài 1b

Phương pháp	Kết quả số	Sai số tuyệt đối	Thời gian chạy (s)
Tổng Riemann	18.171296298989	7.662037034344	39.061605482
Monte Carlo	25.880780693346	0.047447360013	0.337083387

Nhận xét:

- Với cùng bài toán 10 chiều, tổng Riemann cho sai số rất lớn dù thời gian chạy đã lên tới khoảng 39 giây. Nguyên nhân là số điểm trên mỗi chiều chỉ lấy bằng 6, nên lưới còn thưa, trong khi tổng số tổ hợp điểm vẫn rất lớn.
- Monte Carlo với chỉ 10^4 điểm ngẫu nhiên cho kết quả gần nghiệm giải tích hơn nhiều, đồng thời thời gian chạy chỉ khoảng 0.34 giây.
- Bài toán này minh họa rõ nhược điểm của phương pháp Riemann trong không gian nhiều chiều: chi phí tính tăng theo N^D . Ngược lại, Monte Carlo tránh được sự phức tạp trong phép tính khi làm với D chiều với D lớn.

4.3 Thực hành MC 2a: Khối lượng của khối khí trong lập phương đơn vị

Mật độ khối lượng được cho bởi

$$\rho(x, y, z) = x^2 + y^2 + z^2, \quad (x, y, z) \in [0, 1]^3.$$

Khối lượng cần tính là

$$M = \iiint_{[0,1]^3} (x^2 + y^2 + z^2) dx dy dz.$$

Tính giải tích cho kết quả:

$$M = \int_0^1 x^2 dx + \int_0^1 y^2 dy + \int_0^1 z^2 dz = 1.$$

Kết quả Monte Carlo đã chạy:

Bảng 4: Kết quả bài MC 2a

N	Kết quả MC	Sai số ước lượng	Sai số tuyệt đối so với nghiệm đúng
10^6	1.000189365043	0.000516962992	0.000189365043

Kết quả thu được phù hợp với nghiệm đúng $M = 1$. Sai số thực tế nhỏ hơn sai số ước lượng, nên kết quả là hợp lý.

4.4 Thực hành MC 2b: Thể tích phần giao giữa hình cầu và hình trụ

Bài toán yêu cầu tính thể tích của miền

$$\Omega = \{(x, y, z) : x^2 + y^2 + z^2 \leq 1, x^2 + y^2 \leq 0.25\}.$$

Trong chương trình, miền lấy mẫu được chọn là khối lập phương $[-1, 1]^3$ có thể tích bằng 8. Mỗi điểm ngẫu nhiên được kiểm tra xem có thuộc miền Ω hay không. Từ đó, thể tích được xấp xỉ bởi

$$V_{\Omega} \approx 8 \frac{N_{\text{inside}}}{N}.$$

Ngoài Monte Carlo, bài toán này còn có thể kiểm tra lại bằng tích phân giải tích trong tọa độ trụ:

$$V = \int_0^{2\pi} d\phi \int_0^{0.5} r dr \int_{-\sqrt{1-r^2}}^{\sqrt{1-r^2}} dz.$$

Suy ra

$$V = 4\pi \int_0^{0.5} r \sqrt{1-r^2} dr = \frac{4\pi}{3} \left[1 - \left(\frac{3}{4} \right)^{3/2} \right] \approx 1.468091158435.$$

Kết quả Monte Carlo đã chạy:

Bảng 5: Kết quả bài MC 2b

N	Kết quả MC	Nghiệm giải tích	Sai số tuyệt đối
10^6	1.471008000	1.468091158435	0.002916841565

Sai số khoảng 2.9×10^{-3} là chấp nhận được đối với phép tính Monte Carlo với 10^6 điểm ngẫu nhiên trong không gian ba chiều.

5 Kết luận

Từ các kết quả đã tính được, có thể rút ra các kết luận chính sau:

- Phương pháp Monte Carlo cho phép xử lý tích phân nhiều chiều một cách trực tiếp, với thuật toán rất đơn giản và chạy khá ổn định.
- Với bài 1a, kết quả Monte Carlo bám tốt nghiệm giải tích $D/3$. Tuy nhiên, để giảm sai số xuống cỡ 10^{-4} thì số mẫu cần lên đến hàng chục triệu điểm. (Em đã thử test nhưng máy em đã không chạy nổi, dù em đã thử dùng thêm MPI trên server của em nhưng vẫn k ra được 10^{-4})
- Với bài 1b, Monte Carlo vượt trội hơn tổng Riemann trong bài toán 10 chiều xét trên cùng mức độ chính xác thực tế. Tổng Riemann chịu ảnh hưởng rất mạnh của số chiều do số nút lưới tăng theo hàm mũ.
- Với các bài toán hình học và bài toán khối lượng trong MC 2, Monte Carlo cho kết quả phù hợp với nghiệm giải tích hoặc với giá trị kiểm tra độc lập.

Nhìn chung, đối với các bài toán tích phân nhiều chiều trong vật lý tính toán, Monte Carlo là một công cụ hiệu quả hơn các phương pháp tổng Riemann khi số chiều của không gian tích phân tăng cao.